

Connecting Login Route



You said:

ChatGPT said:

Connecting Login Route in Node.js + Express + Passport.js

When a user logs in:

1. User enters username & password.
2. Data goes to `/login` route.
3. Passport checks credentials.
4. If correct → user logged in.
5. If wrong → error message shown.

1. Install Required Packages

```
Bashnpm install passport passport-local passport-local-mongoose express-session connect-flash
```

2. Setup Passport in **app.js**

```
JavaScriptconst express = require("express");
const app = express();

const session = require("express-session");
const flash = require("connect-flash");

const passport = require("passport");
const LocalStrategy = require("passport-local");

const User = require("../models/user");
```

Session Configuration

```
JavaScriptconst sessionOptions = {
  secret: "mysupersecretcode",
  resave: false,
  saveUninitialized: true,
};

app.use(session(sessionOptions));
app.use(flash());
```

3. Initialize Passport

```
JavaScriptapp.use(passport.initialize());
app.use(passport.session());
```

4. Use Local Strategy

```
JavaScriptpassport.use(new LocalStrategy(User.authenticate()));
```

5. Serialize & Deserialize User

```
JavaScriptpassport.serializeUser(User.serializeUser());
passport.deserializeUser(User.deserializeUser());
```

6. Create Login GET Route

This route shows login form.

```
JavaScriptapp.get("/login", (req, res) => {  
  res.render("users/login.ejs");  
});
```

7. Create Login POST Route

This is the main login connection route.

```
JavaScriptapp.post(  
  "/login",  
  passport.authenticate("local", {  
    failureRedirect: "/login",  
    failureFlash: true,  
  }),  
  (req, res) => {  
    req.flash("success", "Welcome back!");  
  
    res.redirect("/listings");  
  }  
);
```

8. Create Login Form

[views/users/login.ejs](#)

```
HTML<h2>Login Form</h2>

<form action="/login" method="POST">

  <input
    type="text"
    name="username"
    placeholder="username"
  />

  <input
    type="password"
    name="password"
    placeholder="password"
  />

  <button>Login</button>

</form>
```

9. How Passport Works Internally

When this runs:

```
JavaScriptpassport.authenticate("local")
```

Passport:

- Reads username/password from form
- Finds user in MongoDB
- Checks hashed password
- Creates session

- Stores user id in session

10. Flow Diagram

```
Login Form
  ↓
POST /login
  ↓
passport.authenticate("local")
  ↓
User verified
  ↓
Session created
  ↓
Redirect to /listings
```

11. Add Logout Route

```
JavaScriptapp.get("/logout", (req, res, next) => {
  req.logout((err) => {
    if (err) {
      return next(err);
    }

    req.flash("success", "Logged out!");

    res.redirect("/login");
  });
});
```

12. Protect Routes (Very Important)

```
JavaScriptconst isLoggedIn = (req, res, next) => {

  if (!req.isAuthenticated()) {
    req.flash("error", "You must be logged in");
    return res.redirect("/login");
  }

  next();
};
```

Use it:

```
JavaScriptapp.get("/listings/new", isLoggedIn, (req, res) => {  
  res.send("Create Listing");  
});
```

13. Important Interview Questions

Q1. Why use Passport.js?

- Easy authentication
- Session support
- Secure login handling

Q2. What is **passport.authenticate()**?

Middleware that verifies user credentials.

Q3. Difference between Authentication & Authorization?

Authentication	Authorization
Who are you?	What can you access?
Login	Permissions

Q4. What is **serializeUser()**?

Stores user ID in session.

Q5. What is **deserializeUser()**?

Gets full user data from stored session ID.

Complete Login Route

```
JavaScriptapp.post(
  "/login",
  passport.authenticate("local", {
    failureRedirect: "/login",
    failureFlash: true,
  }),
  async (req, res) => {

    req.flash("success", "Welcome back!");

    res.redirect("/listings");
  }
);
```

Common Errors

Error 1

Failed to serialize user

Fix

```
JavaScriptpassport.serializeUser(User.serializeUser());
passport.deserializeUser(User.deserializeUser());
```

Error 2

req.flash is not a function

Fix

```
JavaScriptapp.use(flash());
```

Error 3

Login always fails

Fix

Check:

- username field name
- password field name
- User model uses `passport-local-mongoose`

User Model Example

```
JavaScriptconst mongoose = require("mongoose");
const passportLocalMongoose = require("passport-local-mongoose");

const userSchema = new mongoose.Schema({});

userSchema.plugin(passportLocalMongoose);

module.exports = mongoose.model("User", userSchema);
```

You said:

ChatGPT said:

Logout User in Express + Passport.js

Logout means:

- Remove user session
- Destroy login authentication

- User becomes unauthenticated

Basic Logout Route

```
JavaScriptapp.get("/logout", (req, res, next) => {  
  
  req.logout((err) => {  
  
    if (err) {  
      return next(err);  
    }  
  
    req.flash("success", "You are logged out!");  
  
    res.redirect("/listings");  
  });  
  
});
```

Complete Flow

```
User clicks Logout  
↓  
GET /logout  
↓  
req.logout()  
↓  
Session removed  
↓  
User logged out  
↓  
Redirect page
```

Why **next** is Used?

Passport logout can throw error.

```
JavaScript(req, res, next)
```

If error occurs:

```
JavaScriptreturn next(err);
```

Express handles the error safely.

Add Logout Button in Navbar

```
HTML<a href="/logout">Logout</a>
```

OR

```
HTML<form action="/logout" method="GET">
  <button>Logout</button>
</form>
```

Check Login Status

Passport gives:

```
JavaScriptreq.isAuthenticated()
```

Example:

```
JavaScriptif(req.isAuthenticated()) {
  console.log("User logged in");
}
```

Show Login/Logout Dynamically

In navbar EJS:

```
ejs<% if(currUser) { %>

  <a href="/logout">Logout</a>

<% } else { %>

  <a href="/login">Login</a>

<% } %>
```

Send Current User to All Templates

In **app.js**

```
JavaScriptapp.use((req, res, next) => {  
  
  res.locals.currUser = req.user;  
  
  next();  
});
```

Now every EJS file can access:

```
ejscurrUser
```

Full Authentication Setup

Passport Setup

```
JavaScriptconst passport = require("passport");  
const LocalStrategy = require("passport-local");  
const User = require("../models/user");  
  
app.use(passport.initialize());  
app.use(passport.session());  
  
passport.use(new LocalStrategy(User.authenticate()));  
  
passport.serializeUser(User.serializeUser());  
passport.deserializeUser(User.deserializeUser());
```

Login Route

```
JavaScriptapp.post(  
  "/login",  
  passport.authenticate("local", {  
    failureRedirect: "/login",  
    failureFlash: true,  
  }),  
  (req, res) => {  
  
    req.flash("success", "Welcome Back!");  
  
    res.redirect("/listings");  
  }  
);
```

Logout Route

```
JavaScriptapp.get("/logout", (req, res, next) => {  
  
  req.logout((err) => {  
  
    if(err) {  
      return next(err);  
    }  
  
    req.flash("success", "Logged out successfully!");  
  
    res.redirect("/listings");  
  });  
  
});
```

Protect Routes

```
JavaScriptconst isLoggedIn = (req, res, next) => {  
  
  if(!req.isAuthenticated()) {  
  
    req.flash("error", "Please login first!");  
  
    return res.redirect("/login");  
  }  
  
  next();  
};
```

Use:

```
JavaScriptapp.get("/listings/new", isLoggedIn, (req, res) => {  
  
  res.render("listings/new");  
  
});
```

Important Interview Questions

Q1. What does **req.logout()** do?

It removes authenticated user session.

Q2. Difference between Session Destroy & Logout?

Logout

Removes login auth

Session Destroy

Removes full session

Passport method

Express-session method

Q3. What is `req.user`?

Currently logged-in user object.

Common Errors

Error:

```
req.logout is not a function
```

Fix

Passport not initialized correctly.

Add:

```
JavaScriptapp.use(passport.initialize());  
app.use(passport.session());
```

Error:

```
Cannot GET /logout
```

Fix

Create logout route.

```
JavaScriptapp.get("/logout", ...)
```

Modern Passport Logout Syntax

New Passport versions require callback:

```
JavaScriptreq.logout(function(err) {  
  if(err) {  
    return next(err);  
  }  
  res.redirect("/");  
});
```

Old syntax:

```
JavaScriptreq.logout();
```

Old version may not work in Passport 0.6+.

You said:

ChatGPT said:

Add Styling to Login & Logout Pages (EJS + Bootstrap)

1. Add Bootstrap CDN

Inside `views/includes/boilerplate.ejs` or `<head>` section:

```
HTML<link  
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"  
  rel="stylesheet"  
>
```

2. Styled Login Page

`views/users/login.ejs`

```
HTML<div class="container d-flex justify-content-center mt-5">

  <div class="card shadow p-4 login-card">

    <h2 class="text-center mb-4">Login</h2>

    <form action="/login" method="POST">

      <div class="mb-3">

        <label class="form-label">Username</label>

        <input
          type="text"
          name="username"
          class="form-control"
          placeholder="Enter username"
          required
        />

      </div>

      <div class="mb-3">

        <label class="form-label">Password</label>

        <input
          type="password"
          name="password"
          class="form-control"
          placeholder="Enter password"
          required
        />

      </div>

      <button class="btn btn-dark w-100">
        Login
      </button>

    </form>

  </div>

</div>
```

3. Add CSS Styling

Create:

```
public/css/style.css
```

Add:

```
CSSbody {  
  background-color: #f8f9fa;  
}  
  
.login-card {  
  width: 400px;  
  border-radius: 15px;  
}  
  
.card {  
  border: none;  
}
```

4. Connect CSS File

Inside layout/boilerplate:

```
HTML<link rel="stylesheet" href="/css/style.css">
```

5. Serve Static Files

In `app.js`

```
JavaScriptapp.use(express.static(path.join(__dirname, "/public")));
```


6. Styled Navbar with Login & Logout

```
HTML<nav class="navbar navbar-expand-lg navbar-dark bg-dark px-3">

  <a class="navbar-brand" href="/listings">
    WanderLust
  </a>

  <div class="ms-auto">

    <% if(currUser) { %>

      <a href="/logout" class="btn btn-danger">
        Logout
      </a>

    <% } else { %>

      <a href="/login" class="btn btn-light">
        Login
      </a>

    <% } %>

  </div>

</nav>
```

7. Add Flash Message Styling

Inside `boilerplate.ejs`

```

ejs<div class="container mt-3">

  <% if(success && success.length > 0) { %>

    <div class="alert alert-success alert-dismissible fade show">

      <%= success %>

      <button
        type="button"
        class="btn-close"
        data-bs-dismiss="alert">
      </button>

    </div>

  <% } %>

  <% if(error && error.length > 0) { %>

    <div class="alert alert-danger alert-dismissible fade show">

      <%= error %>

      <button
        type="button"
        class="btn-close"
        data-bs-dismiss="alert">
      </button>

    </div>

  <% } %>

</div>

```

8. Add Bootstrap JS

Before `</body>`:

```

HTML<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>

```

9. Add res.locals for Flash

In `app.js`

```
JavaScriptapp.use((req, res, next) => {  
  
  res.locals.success = req.flash("success");  
  res.locals.error = req.flash("error");  
  res.locals.currUser = req.user;  
  
  next();  
});
```

Final Result

You now have:

- Beautiful Login Page
- Styled Buttons
- Flash Messages
- Dynamic Navbar
- Responsive Card Design
- Clean UI using Bootstrap

Extra Modern Styling (Optional)

```
CSS.login-card {  
  backdrop-filter: blur(10px);  
  background: white;  
  transition: 0.3s;  
}  
  
.login-card:hover {  
  transform: translateY(-5px);  
}
```

Interview Questions

Why use Bootstrap?

- Fast UI development
- Responsive design

- Prebuilt components

Why use static folder?

To serve:

- CSS
- JS
- Images
- Fonts

Why use **res.locals**?

To send variables globally to all EJS templates.

You said:

ChatGPT said:

Login Automatically After Signup (Passport.js)

Normally after signup:

1. User registers
2. Redirect to login page
3. User logs in again

But with Passport.js you can:

✅ Signup ✅ Automatically login ✅ Redirect to homepage/dashboard

Complete Signup Route with Auto Login

```
JavaScriptapp.post("/signup", async (req, res) => {

  try {

    let { username, email, password } = req.body;

    const newUser = new User({
      email,
      username,
    });

    const registeredUser = await User.register(
      newUser,
      password
    );

    console.log(registeredUser);

    req.login(registeredUser, (err) => {

      if(err) {
        return next(err);
      }

      req.flash(
        "success",
        "Welcome to WanderLust!"
      );

      res.redirect("/listings");

    });

  } catch(err) {

    req.flash("error", err.message);

    res.redirect("/signup");
  }
});
```

Main Magic Line

```
JavaScriptreq.login(registeredUser, (err) => {
```

This automatically creates login session after signup.

How It Works

```
graph TD; A[User Signup] --> B[User saved in MongoDB]; B --> C[req.login()]; C --> D[Session created]; D --> E[User automatically logged in]; E --> F[Redirect to /listings];
```

Add **next** Parameter (Important)

Better version:

```
JavaScriptapp.post("/signup", async (req, res, next) => {
```

Full route:

```
JavaScriptapp.post("/signup", async (req, res, next) => {  
  
  try {  
  
    let { username, email, password } = req.body;  
  
    const newUser = new User({  
      email,  
      username,  
    });  
  
    const registeredUser = await User.register(  
      newUser,  
      password  
    );  
  
    req.login(registeredUser, (err) => {  
  
      if(err) {  
        return next(err);  
      }  
  
      req.flash(  
        "success",  
        "Welcome to WanderLust!"  
      );  
  
      res.redirect("/listings");  
  
    });  
  
  } catch(err) {  
  
    req.flash("error", err.message);  
  
    res.redirect("/signup");  
  }  
  
});
```

Signup Form Example

```
HTML<form action="/signup" method="POST">

  <input
    type="text"
    name="username"
    placeholder="username"
  />

  <input
    type="email"
    name="email"
    placeholder="email"
  />

  <input
    type="password"
    name="password"
    placeholder="password"
  />

  <button>Signup</button>

</form>
```

Why **User.register()**?

From **passport-local-mongoose**.

It:

- Hashes password
- Stores salt
- Saves user securely

DO NOT DO THIS ❌

```
JavaScriptnewUser.password = password;
await newUser.save();
```

Password will store as plain text.

Correct Method

```
JavaScriptawait User.register(newUser, password);
```

User Model Example

```
JavaScriptconst mongoose = require("mongoose");

const passportLocalMongoose =
  require("passport-local-mongoose");

const userSchema = new mongoose.Schema({

  email: {
    type: String,
    required: true,
  },

});

userSchema.plugin(passportLocalMongoose);

module.exports =
  mongoose.model("User", userSchema);
```

Add Signup Route (GET)

```
JavaScriptapp.get("/signup", (req, res) => {

  res.render("users/signup.ejs");

});
```

Protect Routes After Login

```
JavaScriptconst isLoggedIn = (req, res, next) => {  
  
  if(!req.isAuthenticated()) {  
  
    req.flash(  
      "error",  
      "Please login first!"  
    );  
  
    return res.redirect("/login");  
  }  
  
  next();  
};
```

Interview Questions

Q1. What does **req.login()** do?

Creates login session manually.

Q2. Why use Passport Local Mongoose?

- Password hashing
- Easy authentication
- Session support

Q3. Difference between **req.login()** and **passport.authenticate()**?

req.login()	passport.authenticate()
Manual login	Middleware authentication

req.login() **passport.authenticate()**

Used after signup Used during login

Common Errors

Error

req.login is not a function

Fix

Initialize Passport:

```
JavaScriptapp.use(passport.initialize());  
app.use(passport.session());
```

Error

User.register is not a function

Fix

Add plugin:

```
JavaScriptuserSchema.plugin(passportLocalMongoose);
```

Final Flow

```
graph TD  
  A[Signup Form] --> B[POST /signup]  
  B --> C[User.register()]  
  C --> D[req.login()]  
  D --> E[Session Created]  
  E --> F[Redirect User]
```

You said:

ChatGPT said:

POST Login Route in Express + Passport.js

The POST login route checks username and password entered by the user.

Complete POST Login Route

```
JavaScriptapp.post(
  "/login",

  passport.authenticate("local", {
    failureRedirect: "/login",
    failureFlash: true,
  }),

  async (req, res) => {

    req.flash(
      "success",
      "Welcome back to WanderLust!"
    );

    res.redirect("/listings");
  }
);
```

How It Works

```
graph TD
    A[User fills login form] --> B[POST /login]
    B --> C[passport.authenticate("local")]
    C --> D[Checks username/password]
    D --> E[If correct → login success]
    D --> F[If wrong → redirect login page]
```

Login Form

views/users/login.ejs

```
HTML<div class="container mt-5">

  <div class="row justify-content-center">

    <div class="col-md-5">

      <div class="card shadow p-4">

        <h2 class="text-center mb-4">
          Login
        </h2>

        <form action="/login" method="POST">

          <div class="mb-3">

            <label class="form-label">
              Username
            </label>

            <input
              type="text"
              name="username"
              class="form-control"
              required
            />

          </div>

          <div class="mb-3">

            <label class="form-label">
              Password
            </label>

            <input
              type="password"
              name="password"
              class="form-control"
              required
            />

          </div>

          <button class="btn btn-dark w-100">
            Login
          </button>

        </form>
```

```
</div>

</div>

</div>

</div>
```

Required Passport Setup

In `app.js`

```
JavaScriptconst passport = require("passport");
const LocalStrategy = require("passport-local");
const User = require("../models/user");
```

Initialize Passport

```
JavaScriptapp.use(passport.initialize());

app.use(passport.session());
```

Configure Local Strategy

```
JavaScriptpassport.use(
  new LocalStrategy(User.authenticate())
);
```

Serialize & Deserialize

```
JavaScriptpassport.serializeUser(
  User.serializeUser()
);

passport.deserializeUser(
  User.deserializeUser()
);
```

Flash Message Setup

```
JavaScriptconst flash = require("connect-flash");

app.use(flash());
```

res.locals Setup

```
JavaScriptapp.use((req, res, next) => {

  res.locals.success =
    req.flash("success");

  res.locals.error =
    req.flash("error");

  res.locals.currUser =
    req.user;

  next();
});
```

Flash Message EJS

```
ejs<% if(success.length > 0) { %>

<div class="alert alert-success">
  <%= success %>
</div>

<% } %>

<% if(error.length > 0) { %>

<div class="alert alert-danger">
  <%= error %>
</div>

<% } %>
```


What `passport.authenticate()` Does

```
JavaScriptpassport.authenticate("local")
```

Passport automatically:

- Finds user
- Verifies password
- Creates session
- Stores user ID
- Adds `req.user`

Login Success

After successful login:

```
JavaScriptreq.user
```

Contains logged-in user data.

Check User Login Status

```
JavaScriptreq.isAuthenticated()
```

Example:

```
JavaScriptif(req.isAuthenticated()) {  
    console.log("Logged In");  
}
```

Protect Private Routes

```
JavaScriptconst isLoggedIn = (req, res, next) => {  
  
  if(!req.isAuthenticated()) {  
  
    req.flash(  
      "error",  
      "Please login first!"  
    );  
  
    return res.redirect("/login");  
  }  
  
  next();  
};
```

Use:

```
JavaScriptapp.get(  
  "/listings/new",  
  isLoggedIn,  
  (req, res) => {  
  
    res.render("listings/new");  
  
  });
```

Common Errors

Error

Failed to serialize user

Fix

```
JavaScriptpassport.serializeUser(  
  User.serializeUser()  
);  
  
passport.deserializeUser(  
  User.deserializeUser()  
);
```

Error

Login always fails

Fix

Check:

- Username field name
- Password field name
- Passport configuration

Error

req.flash is not a function

Fix

```
JavaScriptapp.use(flash());
```

Interview Questions

Q1. Why use POST for login?

Because sensitive data should not go in URL.

Q2. What is Local Strategy?

Authentication using username & password.

Q3. What does Passport store in session?

User ID.

Full Login Flow

```
graph TD; A[Login Form] --> B[POST /login]; B --> C[passport.authenticate()]; C --> D[Session Created]; D --> E[req.user available]; E --> F[Redirect User];
```

You said:

ChatGPT said:

Listing Owner in MERN / Express + MongoDB

Listing Owner means:

👉 Every listing belongs to a specific logged-in user.

Example:

- User A creates Listing A
- User B creates Listing B

Only the owner can:

- Edit listing
- Delete listing

1. Add Owner Field in Listing Schema

`models/listing.js`

```
JavaScriptconst mongoose = require("mongoose");

const Schema = mongoose.Schema;

const listingSchema = new Schema({

  title: String,

  description: String,

  price: Number,

  location: String,

  country: String,

  owner: {
    type: Schema.Types.ObjectId,
    ref: "User",
  },

});

module.exports =
mongoose.model("Listing", listingSchema);
```

Why Use **ObjectId**?

MongoDB stores relationship using IDs.

```
Listing
↓
owner → User ID
```

2. Save Logged-in User as Owner

When creating listing:

```
JavaScriptapp.post(
  "/listings",
  isLoggedIn,
  async (req, res) => {

    const newListing =
      new Listing(req.body.listing);

    newListing.owner = req.user._id;

    await newListing.save();

    req.flash(
      "success",
      "New Listing Created!"
    );

    res.redirect("/listings");
  });
```

How It Works

Logged-in User
↓
req.user._id
↓
Saved in owner field

3. Populate Owner Information

Without populate:

```
owner: ObjectId("73737373")
```

With populate:

```
owner: {
  username: "aamir",
  email: "abc@gmail.com"
}
```

Show Route with Populate

```
JavaScriptapp.get("/listings/:id", async (req, res) => {  
  
  const listing =  
    await Listing.findById(req.params.id)  
    .populate("owner");  
  
  res.render(  
    "listings/show.ejs",  
    { listing }  
  );  
  
});
```

4. Show Owner Name in EJS

```
ejs<p>  
  Created By:  
  <%= listing.owner.username %>  
</p>
```

5. Restrict Edit/Delete to Owner

Create middleware:

```
JavaScriptconst isOwner = async (req, res, next) => {  
  
  let { id } = req.params;  
  
  const listing =  
    await Listing.findById(id);  
  
  if(  
    !listing.owner.equals(req.user._id)  
  ) {  
  
    req.flash(  
      "error",  
      "You are not the owner!"  
    );  
  
    return res.redirect(  
      `/listings/${id}`  
    );  
  }  
  
  next();  
};
```

6. Use Middleware

Edit Route

```
JavaScriptapp.get(  
  "/listings/:id/edit",  
  isLoggedIn,  
  isOwner,  
  
  async (req, res) => {  
  
    const listing =  
      await Listing.findById(  
        req.params.id  
      );  
  
    res.render(  
      "listings/edit.ejs",  
      { listing }  
    );  
  });
```


Delete Route

```
JavaScriptapp.delete(  
  "/listings/:id",  
  isLoggedIn,  
  isOwner,  
  
  async (req, res) => {  
  
    await Listing.findByIdAndDelete(  
      req.params.id  
    );  
  
    req.flash(  
      "success",  
      "Listing Deleted!"  
    );  
  
    res.redirect("/listings");  
  });
```

7. Hide Edit/Delete Buttons

Only show buttons to owner.

[show.ejs](#)

```
ejs<% if(currUser &&
listing.owner._id.equals(currUser._id))
{ %>

    <a
      href="/listings/<%= listing._id %>/edit"
      class="btn btn-dark"
    >
      Edit
    </a>

    <form
      method="POST"
      action="/listings/<%= listing._id %>?_method=DELETE"
    >

      <button class="btn btn-danger">
        Delete
      </button>

    </form>

<% } %>
```

8. Add Current User Globally

In `app.js`

```
JavaScriptapp.use((req, res, next) => {

  res.locals.currUser = req.user;

  next();
});
```

Complete Ownership Flow

```
User Login
↓
Create Listing
↓
owner = req.user._id
↓
Save in MongoDB
↓
Owner can edit/delete
Others cannot
```

Example MongoDB Document

```
JSON{
  "_id": "987654",
  "title": "Mountain House",
  "owner": "64738293"
}
```

64738293 = User ID

Interview Questions

Q1. Why use **ref** in MongoDB?

To create relationships between collections.

Q2. What does **populate()** do?

Replaces ObjectId with actual document data.

Q3. Why use **.equals()**?

Because MongoDB ObjectIds are objects.

Wrong ❌

```
JavaScriptlisting.owner == req.user._id
```

Correct ✅

```
JavaScriptlisting.owner.equals(req.user._id)
```

Common Errors

Error

```
Cannot read property '_id' of undefined
```

Fix

Use:

```
JavaScriptisLoggedIn
```

before **isOwner**.

Error

```
listing.owner.equals is not a function
```

Fix

Owner not stored as ObjectId.

Use:

```
JavaScriptowner: {  
  type: Schema.Types.ObjectId,  
  ref: "User"  
}
```

You said:

Starting with authorization

ChatGPT said:

Starting with Authorization in Express + Passport.js

What is Authorization?

Authorization means:

👉 Checking **what a user is allowed to do**.

Example:

Situation	Type
User login	Authentication
Can edit own listing only	Authorization

Authentication vs Authorization

Authentication	Authorization
Who are you?	What can you do?
Login system	Permission system
Uses username/password	Uses ownership/roles

Real Example

Suppose:

- Aamir creates Listing A
- John creates Listing B

Now:

✅ Aamir can edit Listing A ❌ Aamir cannot edit Listing B

This is Authorization.

Step 1: Add Owner in Listing Schema

`models/listing.js`

```
JavaScriptowner: {  
  type: Schema.Types.ObjectId,  
  ref: "User",  
}
```

Complete Example:

```
JavaScriptconst mongoose = require("mongoose");  
  
const Schema = mongoose.Schema;  
  
const listingSchema = new Schema({  
  
  title: String,  
  
  price: Number,  
  
  location: String,  
  
  owner: {  
    type: Schema.Types.ObjectId,  
    ref: "User",  
  },  
  
});  
  
module.exports =  
mongoose.model("Listing", listingSchema);
```

Step 2: Save Current User as Owner

When creating listing:

```
JavaScriptapp.post(
  "/listings",
  isLoggedIn,

  async (req, res) => {

    const newListing =
      new Listing(req.body.listing);

    newListing.owner =
      req.user._id;

    await newListing.save();

    req.flash(
      "success",
      "Listing Created!"
    );

    res.redirect("/listings");
  });
```

Step 3: Create Authorization Middleware

isOwner

```
JavaScriptconst Listing =
require("../models/listing");

module.exports.isOwner =
async (req, res, next) => {

  let { id } = req.params;

  const listing =
    await Listing.findById(id);

  if(
    !listing.owner.equals(req.user._id)
  ) {

    req.flash(
      "error",
      "You are not the owner!"
    );

    return res.redirect(
      `/listings/${id}`
    );
  }

  next();
};
```

Why `.equals()`?

MongoDB ObjectId is object type.

Wrong ❌

```
JavaScriptlisting.owner == req.user._id
```

Correct ✅

```
JavaScriptlisting.owner.equals(req.user._id)
```


Step 4: Use Middleware in Edit Route

```
JavaScriptapp.get(
  "/listings/:id/edit",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    const listing =
      await Listing.findById(
        req.params.id
      );

    res.render(
      "listings/edit.ejs",
      { listing }
    );

  });
```

Step 5: Use Middleware in Delete Route

```
JavaScriptapp.delete(
  "/listings/:id",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    await Listing.findByIdAndDelete(
      req.params.id
    );

    req.flash(
      "success",
      "Listing Deleted!"
    );

    res.redirect("/listings");

  });
```

Authorization Flow

```
graph TD; A[User Login] --> B[Open Edit/Delete Route]; B --> C[isOwner Middleware]; C --> D[Check owner === current user]; D --> E[Allowed OR Denied];
```

Step 6: Hide Buttons from Other Users

show.ejs

```
ejs<% if(currUser &&
listing.owner._id.equals(currUser._id))
{ %>

  <a
    href="/listings/<%= listing._id %>/edit"
    class="btn btn-dark"
  >
    Edit
  </a>

  <form
    method="POST"
    action="/listings/<%= listing._id %>?_method=DELETE"
  >

    <button class="btn btn-danger">
      Delete
    </button>

  </form>

<% } %>
```

Step 7: Send Current User Globally

In `app.js`

```
JavaScriptapp.use((req, res, next) => {  
  
  res.locals.currUser =  
    req.user;  
  
  next();  
});
```

Important Middleware Order

Correct 

```
JavaScriptisLoggedIn,  
isOwner
```

Wrong 

```
JavaScriptisOwner,  
isLoggedIn
```

Because `req.user` exists only after login.

Common Errors

Error

```
Cannot read properties of undefined
```

Cause

`req.user` does not exist.

Fix

Use:

```
JavaScriptisLoggedIn
```

before `isOwner`.

Error

```
listing.owner.equals is not a function
```

Cause

Owner not stored as ObjectId.

Fix

```
JavaScriptowner: {  
  type: Schema.Types.ObjectId,  
  ref: "User"  
}
```

Interview Questions

Q1. What is Authorization?

Checking user permissions/access.

Q2. Difference between Authentication and Authorization?

Authentication = Login Authorization = Permission

Q3. Why use middleware for authorization?

Reusable security logic.

Final Secure Route Example

```
JavaScriptapp.put(
  "/listings/:id",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    await Listing.findByIdAndUpdate(
      req.params.id,
      {...req.body.listing}
    );

    req.flash(
      "success",
      "Listing Updated!"
    );

    res.redirect(
      `/listings/${req.params.id}`
    );
  });
```

You said:

Authorization for listing

ChatGPT said:

Authorization for Listing in Express + MongoDB

Authorization means:

☒ Only the listing owner can:

- Edit listing
- Update listing
- Delete listing

Other users cannot modify someone else's listing.

Step 1: Add Owner Field in Listing Schema

models/listing.js

```
JavaScriptconst mongoose = require("mongoose");

const Schema = mongoose.Schema;

const listingSchema = new Schema({

  title: String,

  description: String,

  price: Number,

  location: String,

  country: String,

  owner: {
    type: Schema.Types.ObjectId,
    ref: "User",
  },

});

module.exports =
mongoose.model("Listing", listingSchema);
```

Why **ref: "User"**?

Creates relationship:

Listing → User

MongoDB stores user ID as owner.

Step 2: Save Current User as Owner

When creating listing:

```
JavaScriptapp.post(
  "/listings",

  isLoggedIn,

  async (req, res) => {

    const newListing =
      new Listing(req.body.listing);

    newListing.owner =
      req.user._id;

    await newListing.save();

    req.flash(
      "success",
      "New Listing Created!"
    );

    res.redirect("/listings");

  });
```

How Ownership Works

Logged-in User
↓
req.user._id
↓
Saved inside owner field

Step 3: Create Authorization Middleware

Create:

middleware.js

isOwner Middleware

```
JavaScriptconst Listing =
require("../models/listing");

module.exports.isOwner =
async (req, res, next) => {

  let { id } = req.params;

  const listing =
    await Listing.findById(id);

  if(
    !listing.owner.equals(req.user._id)
  ) {

    req.flash(
      "error",
      "You are not the owner!"
    );

    return res.redirect(
      `/listings/${id}`
    );
  }

  next();
};
```

Why `.equals()`?

MongoDB ObjectId is an object.

Wrong ❌

```
JavaScriptlisting.owner == req.user._id
```

Correct ✅

```
JavaScriptlisting.owner.equals(req.user._id)
```


Step 4: Use Middleware in Edit Route

```
JavaScriptapp.get(
  "/listings/:id/edit",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    const listing =
      await Listing.findById(
        req.params.id
      );

    res.render(
      "listings/edit.ejs",
      { listing }
    );

  });
```

Step 5: Use Middleware in Update Route

```
JavaScriptapp.put(
  "/listings/:id",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    await Listing.findByIdAndUpdate(
      req.params.id,
      { ...req.body.listing }
    );

    req.flash(
      "success",
      "Listing Updated!"
    );

    res.redirect(
      `/listings/${req.params.id}`
    );

  });
```

Step 6: Use Middleware in Delete Route

```
JavaScriptapp.delete(
  "/listings/:id",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    await Listing.findByIdAndDelete(
      req.params.id
    );

    req.flash(
      "success",
      "Listing Deleted!"
    );

    res.redirect("/listings");

  });
```

Step 7: Populate Owner Data

Without populate:

```
owner: ObjectId("737373")
```

With populate:

```
owner: {
  username: "aamir"
}
```

Show Route

```
JavaScriptapp.get(
  "/listings/:id",

  async (req, res) => {

    const listing =
      await Listing.findById(
        req.params.id
      ).populate("owner");

    res.render(
      "listings/show.ejs",
      { listing }
    );

  });
```

Step 8: Show Owner Name

show.ejs

```
ejs<p>
  Created By:
  <%= listing.owner.username %>
</p>
```

Step 9: Hide Edit/Delete Buttons

Only owner sees buttons.

```
ejs<% if(currUser &&
listing.owner._id.equals(currUser._id))
{ %>

    <a
      href="/listings/<%= listing._id %>/edit"
      class="btn btn-dark"
    >
      Edit
    </a>

    <form
      method="POST"
      action="/listings/<%= listing._id %>?_method=DELETE"
    >

      <button class="btn btn-danger">
        Delete
      </button>

    </form>

<% } %>
```

Step 10: Send Current User Globally

In `app.js`

```
JavaScriptapp.use((req, res, next) => {

  res.locals.currUser =
    req.user;

  next();
});
```

Authorization Flow

```
graph TD; A[User Login] --> B[Create Listing]; B --> C["owner = req.user._id"]; C --> D[Edit/Delete Request]; D --> E[isOwner Middleware]; E --> F[Allowed or Denied];
```

Important Middleware Order

Correct 

```
JavaScriptisLoggedIn,  
isOwner
```

Wrong 

```
JavaScriptisOwner,  
isLoggedIn
```

Common Errors

Error

```
Cannot read properties of undefined
```

Cause

`req.user` does not exist.

Fix

Use:

```
JavaScriptisLoggedIn
```

before **isOwner**.

Error

```
listing.owner.equals is not a function
```

Fix

Owner must be:

```
JavaScripttype: Schema.Types.ObjectId
```

Interview Questions

Q1. What is Authorization?

Checking user permissions.

Q2. Why use middleware for authorization?

Reusable security logic.

Q3. What does **populate()** do?

Replaces ObjectId with full document data.

Final Secure Route Example

```
JavaScriptapp.put(
  "/listings/:id",

  isLoggedIn,

  isOwner,

  async (req, res) => {

    await Listing.findByIdAndUpdate(
      req.params.id,
      req.body.listing
    );

    req.flash(
      "success",
      "Listing Updated!"
    );

    res.redirect(
      `/listings/${req.params.id}`
    );

  });
```

You said:

Authorization for reviews

ChatGPT said:

Authorization for Reviews in Express + MongoDB

Review Authorization means:

✓ Only the review owner can:

- Delete review

- Edit review

Other users cannot modify someone else's review.

Step 1: Add Author Field in Review Schema

models/review.js

```
JavaScriptconst mongoose = require("mongoose");

const Schema = mongoose.Schema;

const reviewSchema = new Schema({

  comment: String,

  rating: Number,

  author: {
    type: Schema.Types.ObjectId,
    ref: "User",
  },

});

module.exports =
mongoose.model("Review", reviewSchema);
```

Why Add **author**?

To know:

Which user created the review

Step 2: Save Logged-in User as Author

When creating review:

```
JavaScriptapp.post(
  "/listings/:id/reviews",

  isLoggedIn,

  async (req, res) => {

    let listing =
      await Listing.findById(
        req.params.id
      );

    let newReview =
      new Review(req.body.review);

    newReview.author =
      req.user._id;

    listing.reviews.push(newReview);

    await newReview.save();

    await listing.save();

    req.flash(
      "success",
      "Review Added!"
    );

    res.redirect(
      `/listings/${listing._id}`
    );

  });
```

Flow

```
Logged-in User
↓
req.user._id
↓
Saved in review.author
```

Step 3: Create Review Authorization Middleware

Create inside:

```
middleware.js
```

isReviewAuthor

```
JavaScriptconst Review =  
require("../models/review");  
  
module.exports.isReviewAuthor =  
async (req, res, next) => {  
  
  let { reviewId, id } = req.params;  
  
  let review =  
    await Review.findById(reviewId);  
  
  if(  
    !review.author.equals(req.user._id)  
  ) {  
  
    req.flash(  
      "error",  
      "You are not the review author!"  
    );  
  
    return res.redirect(  
      `/listings/${id}`  
    );  
  }  
  
  next();  
};
```

Why `.equals()`?

MongoDB ObjectIds are objects.

Wrong ❌

```
JavaScriptreview.author == req.user._id
```

Correct ✅

```
JavaScriptreview.author.equals(req.user._id)
```

Step 4: Use Middleware in Delete Route

```
JavaScriptapp.delete(
  "/listings/:id/reviews/:reviewId",

  isLoggedIn,

  isReviewAuthor,

  async (req, res) => {

    let { id, reviewId } =
      req.params;

    await Listing.findByIdAndUpdate(
      id,
      {
        $pull: {
          reviews: reviewId,
        },
      },
    );

    await Review.findByIdAndDelete(
      reviewId
    );

    req.flash(
      "success",
      "Review Deleted!"
    );

    res.redirect(`/listings/${id}`);

  });
```

Step 5: Populate Review Author

Without populate:

```
author: ObjectId("727272")
```

With populate:

```
author: {  
  username: "aamir"  
}
```

Show Route with Populate

```
JavaScriptapp.get(  
  "/listings/:id",  
  
  async (req, res) => {  
  
    let listing =  
      await Listing.findById(  
        req.params.id  
      )  
      .populate({  
        path: "reviews",  
        populate: {  
          path: "author",  
        },  
      })  
      .populate("owner");  
  
    res.render(  
      "listings/show.ejs",  
      { listing }  
    );  
  
  });
```

Step 6: Show Author Name

`show.ejs`

```
ejs<p>  
  Reviewed By:  
  <%= review.author.username %>  
</p>
```

Step 7: Show Delete Button Only to Author

```
ejs<% if(currUser &&
review.author._id.equals(currUser._id))
{ %>

<form
  method="POST"
  action="/listings/<%= listing._id %>/reviews/<%= review._id %>?_method=DELETE"
>

  <button class="btn btn-sm btn-danger">
    Delete
  </button>

</form>

<% } %>
```

Step 8: Send Current User Globally

In `app.js`

```
JavaScriptapp.use((req, res, next) => {

  res.locals.currUser =
    req.user;

  next();
});
```

Review Authorization Flow

```
User Login
↓
Create Review
↓
author = req.user._id
↓
Delete Request
↓
isReviewAuthor Middleware
↓
Allowed or Denied
```

Important Middleware Order

Correct 

```
JavaScriptisLoggedIn,  
isReviewAuthor
```

Wrong 

```
JavaScriptisReviewAuthor,  
isLoggedIn
```

Common Errors

Error

```
Cannot read properties of undefined
```

Cause

`req.user` does not exist.

Fix

Use:

```
JavaScriptisLoggedIn
```

before `isReviewAuthor`.

Error

```
review.author.equals is not a function
```

Fix

Use:

```
JavaScriptauthor: {  
  type: Schema.Types.ObjectId,  
  ref: "User"  
}
```

Interview Questions

Q1. What is review authorization?

Only review creator can modify/delete review.

Q2. Why use middleware?

Reusable security logic.

Q3. Why use populate?

To fetch complete author data from ObjectId.

Final Secure Delete Route

```
JavaScriptapp.delete(
  "/listings/:id/reviews/:reviewId",

  isLoggedIn,

  isReviewAuthor,

  async (req, res) => {

    let { id, reviewId } =
      req.params;

    await Listing.findByIdAndUpdate(
      id,
      {
        $pull: {
          reviews: reviewId,
        },
      }
    );

    await Review.findByIdAndDelete(
      reviewId
    );

    req.flash(
      "success",
      "Review Deleted!"
    );

    res.redirect(`/listings/${id}`);

  });
```